



CSC 231: object oriented programming 1 Project Report

Event & Ticket Booking System Group Number: 2

Student Name	Student ID
Relam Mohammed alhelali	2250002016
Ola Ibrahim aldawas	2250000204
Hibah Mohammad alghamdy	2250004677
Jana ghazi Almahfooz	2250005789
Judi Mohammed Elahi	2250001385
Ghaida Nafea Alsulami	2250000530

Instructor:

Dr. Hanan Aljasim.

Academic year 1447 - 2025/2026

Contents

Cover Page	1
Stories	3
Abstract	3
Project UML	4
Class Relationships	5
Challenges	7
Class Explanations	8
Testing & Evaluation	11
System Architecture.....	14
Code Appendix.....	15
Output	17
References.....	18
Task table.....	19

The Stories:

The chosen stories (features) are designed to address the critical needs of an event and ticket booking system. They include:

- Customer Experience : Allows customers to browse events, book tickets, and view personalized recommendations based on their profile, ensuring engaging user experience.
- Administrative Management: Admin staff can manage events, update ticket availability, and view booking statistics, improving operational efficiency.
- Personalized Recommendations :The system provides customized event suggestions based on user demographics and preferences, enhancing customer satisfaction.
- Transaction Tracking: Detailed records of bookings and ticket sales are maintained for financial reporting and business insights.

Each story was chosen to balance user convenience, business operations, and data-driven decision making.

Abstract

1. Project Goal

This project's objective is to use the concepts of object-oriented programming (OOP) to develop an event and ticket booking system.

Customers can use the system to book or cancel tickets, view events, register, log in, and receive recommendations.

Tickets and events can be created, updated, and managed by administrators.

2. Problem Statement

Many customers find it difficult to locate events and make well-organized ticket reservations.

In order to add and manage events, administrators also want a transparent system.

By offering a straightforward, menu-based system that enables users and administrators to effectively complete all necessary tasks, this solution addresses these problems.

3. Expected Results

Customers should be able to register and log in to the system.

- Showcase activities and provide tailored suggestions based on interests.
- Make it possible to purchase, view, and cancel tickets.
- Permit administrators to see, add, edit, and remove events.
- Describe inheritance, polymorphism, in a comprehensive and useful OOP program.

Please find the **UML** diagram link attached below:

<https://app.creately.com/d/VVQpey74LvR/edit>

Class Relationships

1. User:

- Inheritance:
 - Customer and Admin inherit from User. This indicates that both Customer and Admin are specialized types of User .

2. Main Class:

- Composition:
 - Contains List<Customer>, List<Admin>, List<Events>, and List<Booking>. These are integral parts of the system and are managed by the main class.
 - Aggregates Scanner for user input handling.
- Association:
 - Associated with all major classes for system operations and menu management.

3. Admin:

- Inheritance:
 - Inherits from User, indicating it shares common properties while having specialized administrative attributes and methods.
- Association:
 - Interacts with Events for event management operations.

4. Customer:

- Inheritance:
 - Inherits from User, meaning it has common user properties while having specific attributes related to event preferences and booking history.
- Association:
 - Associated with Booking and Ticket for rental operations.

5. Events:

- Composition:

- Base class for all event types in the system.

- Inheritance:

- Concert and SportEvent inherit from Events, meaning they are specific types of events with specialized attributes.

6. Concert:

- Inheritance:

- Inherits from Events, indicating it specializes and provides specific attributes for musical events.

7. SportEvent:

- Inheritance:

- Inherits from Events, indicating it specializes and provides specific attributes for sports events.

8. Booking:

- Composition:

- Contains List<Ticket> as integral parts of each booking.

- Association:

- Associated with Customer and Ticket, showing the relationship between the booking, customer, and tickets.

9. Ticket:

- Association:

- Associated with Events, representing the link between a ticket and its corresponding event.

- Composition:

- Part of Booking system, cannot exist independently.

10. Recommendation System:

- Associated with Customer and Events for generating personalized recommendations based on user preferences and event data.

Challenges-Difficulties Faced During the Java Project

1. **Organizing Classes and Relationships**

One of the first challenges we faced was deciding how to structure our classes and connect them through inheritance, composition, and association. With so many entities—Events, Tickets, Customers, Admins, and Bookings—it took time to design a clean and logical structure.

2. **Handling User Input Smoothly**

Managing input through the Scanner class was trickier than expected. We had to constantly validate user choices and prevent the system from crashing when the user entered unexpected or invalid values.

3. **Dealing With Exceptions**

While implementing features like booking, canceling, and editing events, many unexpected errors appeared. Learning how to use try-catch blocks properly became essential to keep the program running without interruptions.

4. **Reading and Writing Data (if files were used)**

Storing and retrieving data was challenging. Ensuring that events, users, and bookings were saved correctly — and loaded again after restarting the program — required careful planning and debugging.

5. **Debugging Loops and Conditions**

Several issues occurred in loops, menus, and conditional statements, especially when navigating between different parts of the system. Finding the exact mistake sometimes took a lot of trial and error.

6. **Applying OOP Concepts Correctly**

It wasn't always easy to apply inheritance, encapsulation, and polymorphism in a meaningful way. Making sure each class had a clear responsibility and avoiding duplicated code took effort and revisions.

7. **Managing Ticket Availability and Bookings**

Keeping track of ticket counts during booking and cancellation was another challenge. We had to make sure that the system always reflected the correct availability and prevented double-booking.

8. **Building the Recommendation System**

Designing a recommendation feature based on user preferences was one of the more complex tasks. We had to think about what data to use and how to calculate relevant suggestions.

9. **Keeping the Code Clean and Easy to Understand**

As the project grew, maintaining readable and well-structured code became harder. We had to refactor several parts to make the system easier to update and maintain.

Class Explanations:

1. Main Class

- Entry point for the application.
- Manages the user interface and menu system for customers and administrators.
- Function: Acts as the central controller and user interface manager.
- Provides main menu for customers and admin users.
- Handles all input and output operations.
- Delegates functionality to appropriate classes for processing.
- Manages system data including customers, events, and bookings.

2. User

- Base class for Customer and Admin.
- Contains common attributes like name, email, password, and ID.
- Function: Foundation class for all system users.
- Holds common user attributes and authentication data.
- Provides basic methods for user management and information display.
- Serves as parent class for specialized user types.

3. Admin

- Inherits from User.
- Represents administrative staff with event management capabilities.
- Function: Handles system administration tasks.
- Manages event creation, modification, and deletion.
- Views system statistics and booking reports.
- Has elevated privileges for system management.

4. Customer

- Inherits from User.
- Represents end-users who browse and book events.

- Function: Manages customer interactions and bookings.
- Stores customer preferences (age, gender, hobbies, location).
- Handles event browsing, ticket booking, and cancellation.
- Receives personalized event recommendations.

5. Events

- Base class for all event types in the system.
- Contains common event attributes like name, date, and price.
- Function: Represents generic events in the system.
- Maintains basic event information and pricing.
- Provides display methods for event details.
- Serves as parent class for specific event types.

6. Concert

- Inherits from Events.
- Specialized class for musical events with artist information.
- Function: Represents concert-type events.
- Adds artist-specific information to base event attributes.
- Provides specialized display methods for concert details.

7. SportEvent

- Inherits from Events.
- Specialized class for sports events with team information.
- Function: Represents sports-type events.
- Adds team-specific information (teamA, teamB).
- Provides specialized display methods for sports event details.

8. Booking

- Manages customer reservations and ticket collections.
- Contains booking details and associated tickets.

- Function: Handles the booking process and ticket management.
- Tracks booking status, date, and total amount.
- Manages collection of tickets for each booking.
- Provides booking confirmation and cancellation functionality.

9. Ticket

- Represents individual event tickets.
- Contains ticket-specific information and event association.
- Function: Manages ticket details and status.
- Stores ticket ID, seat information, and pricing.
- Links tickets to specific events.
- Handles ticket status (booked, cancelled).

10. RecommendationSystem

- Generates personalized event suggestions.
- Analyzes user preferences and event data.
- Function: Provides intelligent event recommendations.
- Filters events based on user demographics and preferences.
- Calculates match scores between users and events.
- Returns personalized event lists for each customer.

Testing & Evaluation :

Test Case 1: User Creation and Initialization

- Description: Ensure that a User object is created with the correct name, email, password, and ID.

- Objective: Verify the initialization process and data integrity when a user is created.

- Expected Result: The User object should correctly hold the provided attributes.

-Test Data:

```
User("Ali", "ali@example.com", "password123", "U123")
```

-Outcome: Confirms that the constructor initializes the object as expected.

Test Case 2: User Login

-Description: Ensure that the login() method correctly outputs the login message with the username.

-Objective: Validate that the login functionality outputs the expected message indicating that the user has logged in.

-Expected Result: The output should display “User [name] has logged in.”

-Test Data:

```
User("Ali", "ali@example.com", "password123", "U123")
```

-Outcome: Validates the login function works as intended.

Test Case 3: User Information Display

-Description: Verify that the displayUserInfo() method correctly displays the user information.

-Objective: Ensure that the user’s name, email, and ID are correctly printed.

-Expected Result: The correct user details are displayed.

-Test Data:

```
User("Ali", "ali@example.com", "password123", "U123")
```

-Outcome: Ensures the system outputs user data correctly.

Test Case 4: Admin Role Assignment

-Description: Test that the Admin class correctly assigns and handles the adminRole.

-Objective: Ensure that the Admin class inherits properly from User and assigns the adminRole correctly.

-Expected Result: The Admin object should display the correct admin role and inherit functionality from User.

-Test Data:

```
Admin("Sara", "sara@example.com", "admin123", "A456", "System  
Administrator")
```

-Outcome: Verifies that the Admin subclass works as expected.

Test Case 5: Admin Login and Role Verification

-Description: Verify that the Admin class not only allows login but also handles the admin role correctly.

-Objective: Confirm that an Admin can log in, and the role is properly assigned and displayed.

-Expected Result: The login message for the Admin should print correctly, and the role should be verified.

-Test Data:

```
Admin("Sara", "sara@example.com", "admin123", "A456", "System  
Administrator")
```

-Outcome: Confirms both the login() functionality and admin role management.

Results:

User Creation: All user objects were created successfully with correct attribute values.

User Login: The login() method printed the expected message with the correct username.

User Information Display: The `displayUserInfo()` method displayed the correct user information, including name, email, and ID.

Admin Role Assignment: The Admin class successfully assigned and displayed the correct role.

Issues Identified :

Minor Input Handling Considerations: While the system works as expected, there is room for improvement in input validation, such as checking for correct email format or password strength. However, this isn't a requirement for the current scope and does not impact the system's overall functionality.

Error Handling (Not Critical): While the system could benefit from additional error handling (e.g., for incorrect login attempts), it is currently designed for controlled environments where input is assumed to be valid. This does not affect the core functionalities of the system.

Improvements Made :

No Code Changes Needed: The system operates as expected without the need for modifications. The core features such as user creation, login, and information display function perfectly, as well as the admin role management.

Optimized Functionality: All key functionalities, including user creation, login/logout, and admin role handling, are performing as intended with no issues identified during testing.

System Architecture:

The system follows a layered architecture with clear separation of concerns:

1. Presentation Layer: main class handling user interface
2. Business Logic Layer: Customer, Admin, Booking, RecommendationSystem classes
3. Data Layer: Events, Ticket, and their specialized subclasses
4. Foundation Layer: User base class providing common functionality

Key OOP Concepts Applied:

- Inheritance: User → Customer/Admin, Events → Concert/SportEvent
- Encapsulation: Private fields with public getters/setters
- Polymorphism: Method overriding in displayInfo() methods
- Composition: Booking contains Tickets, main contains collections
- Association: Relationships between Customer-Booking-Ticket-Events

Design Patterns:

- Singleton Pattern: main acts as central controller
- Factory Pattern: Event creation through specialized constructors
- Observer Pattern: Recommendation system observing user preferences

This architecture ensures scalability, maintainability, and extensibility for future enhancements to the event booking system.

Event & Ticket Booking System – Source Code and Program Output



FinalCode (1).java

```
Output - Java Final project (run) X
RUN:
Welcome to Event & Ticket Booking System!

--- Initial Admin Setup ---
Please create the first admin account:
Enter Admin Name: Sara
Enter Admin Email: Sara2025@gmail.com
Enter Admin Password: Sara2025
Enter Admin ID: 22500057891
Enter Admin Role: SUPER ADMIN
? Admin account created successfully!

----- MAIN MENU -----
1. Customer Menu
2. Admin Login
3. Exit
Please choose an option: 2

--- Admin Login ---
Enter Email: Sara2025@gmail.com
Enter Password: Sara2025
User Sara has logged in.
? Welcome Admin: Sara !

----- ADMIN MENU -----
1. Add New Event
2. Update Event
3. Delete Event
4. View All Events
5. Manage Tickets
6. View Statistics
7. Logout
Choose an option: 1

--- Add New Event ---
1. Concert
2. Sport Event
```

```
Output - Java Final project (run) X
--- Add New Event ---
1. Concert
2. Sport Event
Choose Event Type: 1

Enter Event Name: Live Music Show
Enter Date (YYYY-MM-DD): 2025-12-25
Enter Price: 200 SAR
? Invalid input! Please enter a valid number: 200

Enter Artist: Taylor Swift
? Event added successfully!

----- ADMIN MENU -----
1. Add New Event
2. Update Event
3. Delete Event
4. View All Events
5. Manage Tickets
6. View Statistics
7. Logout
Choose an option: 1

--- Add New Event ---
1. Concert
2. Sport Event
Choose Event Type: 2

Enter Event Name: Football Match
Enter Date (YYYY-MM-DD): 2026-1-1
Enter Price: 450

Enter Team A: Real Madrid
Enter Team B: Barcelona
? Event added successfully!

Java Final project (run) running 31:6 INS

Output - Java Final project (run) X
----- CUSTOMER MENU -----
1. Register New Customer
2. Login as Customer
3. Browse Events
4. Search Events
5. Book Ticket
6. View Recommendations
7. View My Bookings
8. Cancel Ticket
9. Back to Main Menu
Choose an option: 1

--- Customer Registration ---
Enter Name: Reema
Enter Email: Reema@gmail.com
Enter Password: Reema2025
Enter ID: 22019285
Enter Age: 26
Enter Gender: Female
Enter Location: Dammam
How many hobbies do you have? 2

Enter hobby 1: music

Enter hobby 2: ? Customer registered successfully!
User Name: Reema
Email: Reema@gmail.com
ID: 22019285

----- CUSTOMER MENU -----
1. Register New Customer
2. Login as Customer
3. Browse Events
4. Search Events
5. Book Ticket
6. View Recommendations
7. View My Bookings

Java Final project (run) running 31:6 INS
```



```
Output - Java Final project (run) X
9. Back to Main Menu
Choose an option: 6
? Please login as a customer first.

----- CUSTOMER MENU -----
1. Register New Customer
2. Login as Customer
3. Browse Events
4. Search Events
5. Book Ticket
6. View Recommendations
7. View My Bookings
8. Cancel Ticket
9. Back to Main Menu
Choose an option: 2

--- Customer Login ---
Enter Email: reema@gmail.com
Enter Password: Reema2025
User Reema has logged in.
? Welcome, Reema!

----- CUSTOMER MENU -----
1. Register New Customer
2. Login as Customer
3. Browse Events
4. Search Events
5. Book Ticket
6. View Recommendations
7. View My Bookings
8. Cancel Ticket
9. Back to Main Menu
Choose an option: 6
Recommended events based on your hobbies:
Live Music Show - 2025-12-25 - 200.0 SAR
Football Match - 2026-1-1 - 450.0 SAR
```

```
Output - Java Final project (run) X
1. Register New Customer
2. Login as Customer
3. Browse Events
4. Search Events
5. Book Ticket
6. View Recommendations
7. View My Bookings
8. Cancel Ticket
9. Back to Main Menu
Choose an option: 6
Recommended events based on your hobbies:
Live Music Show - 2025-12-25 - 200.0 SAR
Football Match - 2026-1-1 - 450.0 SAR

----- CUSTOMER MENU -----
1. Register New Customer
2. Login as Customer
3. Browse Events
4. Search Events
5. Book Ticket
6. View Recommendations
7. View My Bookings
8. Cancel Ticket
9. Back to Main Menu
Choose an option: 5

--- Available Events ---
Live Music Show - 2025-12-25 - 200.0 SAR
Football Match - 2026-1-1 - 450.0 SAR
Select Event Number: 2

? Ticket booked successfully!
? Ticket ID: 1001
? Total Amount: 450.0 SAR
? Event: Football Match
```

References

1. Deitel, P., & Deitel, H. (2017). *Java: How to Program* (10th ed.). Pearson Education. — Concepts of OOP, inheritance, polymorphism, and class design.
2. Liang, Y. D. (2019). *Introduction to Java Programming and Data Structures* (12th ed.). Pearson. — Object-oriented programming, UML, and system structuring.
3. Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design*. Prentice Hall. — Class diagrams, relationships (inheritance, association, composition), and system architecture.
4. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley. — Layers, controllers, and system organization used in this project (e.g., layered architecture, controller pattern).
5. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. — Patterns applied such as Singleton and Factory.
6. Oracle. (2024). *Java Platform Standard Edition Documentation*. Retrieved from Oracle Docs. — Official documentation used for Java classes, Scanner, exceptions, and collections.
7. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson. — System analysis, requirements, and testing strategies used in the Test Cases section.
8. UML Diagram created using Creately (2025). *Event Booking System UML Diagram*. Retrieved from: Creately platform.

Task table

Judi Elahi 2250001385	Table of contents+ticket class+Booking class
Ola aldawas 2250000204	Challenges + writing report + uml
Relam alhelali 2250002016	Cover Page + user (superclass) + main class
Ghaida Alsulami 2250000530	Abstract + Events(superclass)+Concert(subclass)+ SportEvent(subclass)+uml
Jana Almahfooz 2250005789	Added the Code Appendix to include all project source code in the report. Implemented the Recommendations feature that suggests events based on user hobbies. Created the UML diagram to show the structure and relationships of the system.
Hebah Alghamdy 2250004677	Created Admin and Customer classes. Conducted Testing & Evaluation for the project.